



Universidad de Jaén

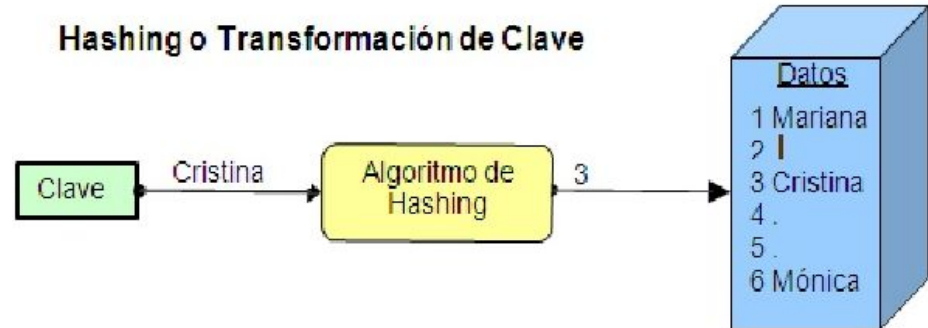
Estructuras de Datos

Dispersión cerrada. Resolución de colisiones

Lección 15:

- Dispersión cerrada
- Tablas con cubetas
- Conjuntos y mapas dispersos

Hashing o Transformación de Clave



Parte I: Funciones de dispersión cerrada

- Motivación
- Funciones de dispersión cerrada
- Operaciones en dispersión cerrada

Motivación

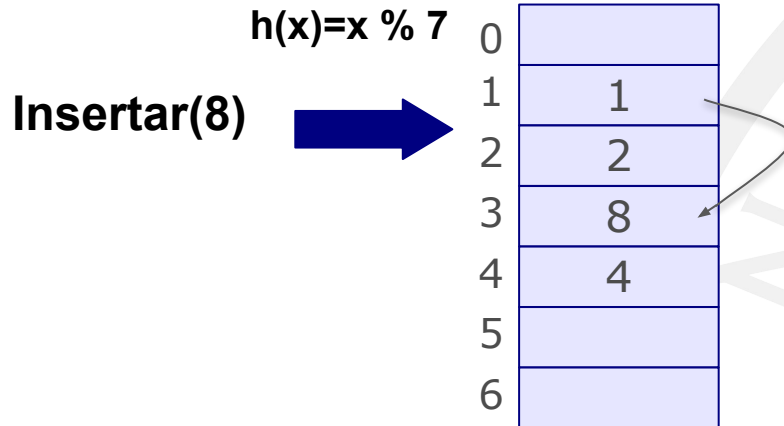
Un fichero disperso es una tabla de dispersión guardada en un fichero en disco. Idealmente este tipo de ficheros permite localizar un dato a partir de su clave con un único acceso.

El encadenamiento separado es una técnica compleja y poco eficiente en disco, al tener que mantener múltiples listas enlazadas.

En este tipo de aplicaciones los datos deben ser guardados únicamente en la tabla de dispersión principal.

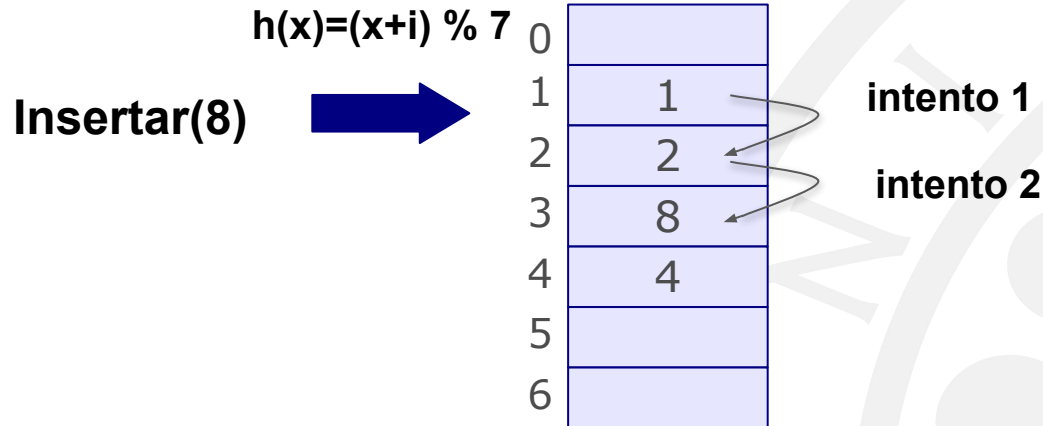
Dispersión cerrada

- Consiste en asignar otra posición libre alternativa de la tabla a una clave cuya posición natural está ya ocupada
- Ventajas
 - Se evitan estructuras secundarias
 - Se aprovecha mejor el espacio de la tabla



Exploración lineal

- Busca un hueco libre en posiciones consecutivas:
 - $h(x) = (x + i) \% t$, siendo i el número de intento (0 inicialmente)



Agrupamientos primarios

- La exploración lineal tiende a generar agrupamientos que atrapan nuevos datos y tienden a crecer
- Son los **agrupamientos primarios** → datos que van a distintas posiciones ($h(x1) \neq h(x2)$) compiten por los mismos huecos durante la resolución de las colisiones

Insertar(1)

0	
1	1
2	
3	
4	
5	
6	

Insertar(2)

0	
1	1
2	2
3	
4	
5	
6	

Insertar(8)

0	
1	1
2	2
3	8
4	
5	
6	

Insertar(9)

0	
1	1
2	2
3	8
4	9
5	
6	

Insertar(15)

0	
1	1
2	2
3	8
4	9
5	15
6	

Agrupamientos secundarios

- Los **agrupamientos secundarios** se dan cuando dos datos distintos x_1 y x_2 van a la misma posición $h(x_1)=h(x_2)$ y por tanto compiten por los mismos huecos

Insertar(1)

0	
1	1
2	
3	
4	
5	
6	

Insertar(8)

0	
1	1
2	8
3	
4	
5	
6	

Insertar(15)

0	
1	1
2	8
3	15
4	
5	
6	

Insertar(22)

0	
1	1
2	8
3	15
4	22
5	
6	

Agrupamientos y rendimiento

- Los dos tipos de agrupamiento tienden a alejar los datos de su posición natural
- Al crecer los agrupamientos se rellenan zonas contiguas de memoria y aumenta la posibilidad de que ocurran colisiones, por lo que el rendimiento de la tabla baja notablemente
- Por ello la exploración lineal no es una buena solución

Exploración cuadrática

- Utiliza la función:
 - $h(x) = (x + i^2) \% t$
- Evita los agrupamientos primarios, pero no los secundarios

Insertar(1)

0	
1	1
2	
3	
4	
5	
6	

Insertar(2)

0	
1	1
2	2
3	
4	
5	
6	

Insertar(8)

0	
1	1
2	2
3	
4	
5	8
6	



Insertar(9)

0	
1	1
2	2
3	9
4	
5	8
6	



Dispersión doble

- La dispersión doble utiliza una segunda función de dispersión para resolver las colisiones
 - $h(x) = (h_1(x) + i \cdot h_2(x)) \% t$
- $h_2(x)$ no debe valer nunca 0. Ejemplos:
 - $h_2(x) = 1 + (x \% q)$ con q primo $< t$
 - $h_2(x) = q - (x \% q)$ con q primo $< t$
- Con $h_2(x)$ bien escogida, se evitan los agrupamientos primarios y secundarios

Dispersión doble

$$h_1(x) = x \% 7$$

$$h_2(x) = x \% 5$$

$$h(x) = ((x \% 7) + i * (5 - x \% 5)) \% 7$$

Insertar(1)

0	
1	1
2	
3	
4	
5	
6	

Insertar(2)

0	
1	1
2	2
3	
4	
5	
6	

Insertar(8)

0	
1	1
2	2
3	8
4	
5	
6	

Insertar(9)

0	
1	1
2	2
3	8
4	9
5	
6	

Insertar(15)

0	
1	1
2	2
3	8
4	9
5	
6	15

Operaciones en dispersión cerrada

- El hecho de que un dato pueda no estar en su posición natural complica las búsquedas y borrados
- Al buscar hay que continuar hasta que se encuentra el dato o se llega a un hueco

$$h(x) = (x + i) \% 7$$

Buscar(8)



0	
1	1
2	2
3	8
4	9
5	
6	

Buscar(15)



0	
1	1
2	2
3	8
4	9
5	⊘
6	

Problemas de los borrados

- Sin embargo, ¡un borrado puede confundir la búsqueda!

$$h(x) = (x + i) \% 7$$

0	
1	1
2	2
3	8
4	9
5	
6	

Borrar(2)



0	
1	1
2	X
3	8
4	9
5	
6	

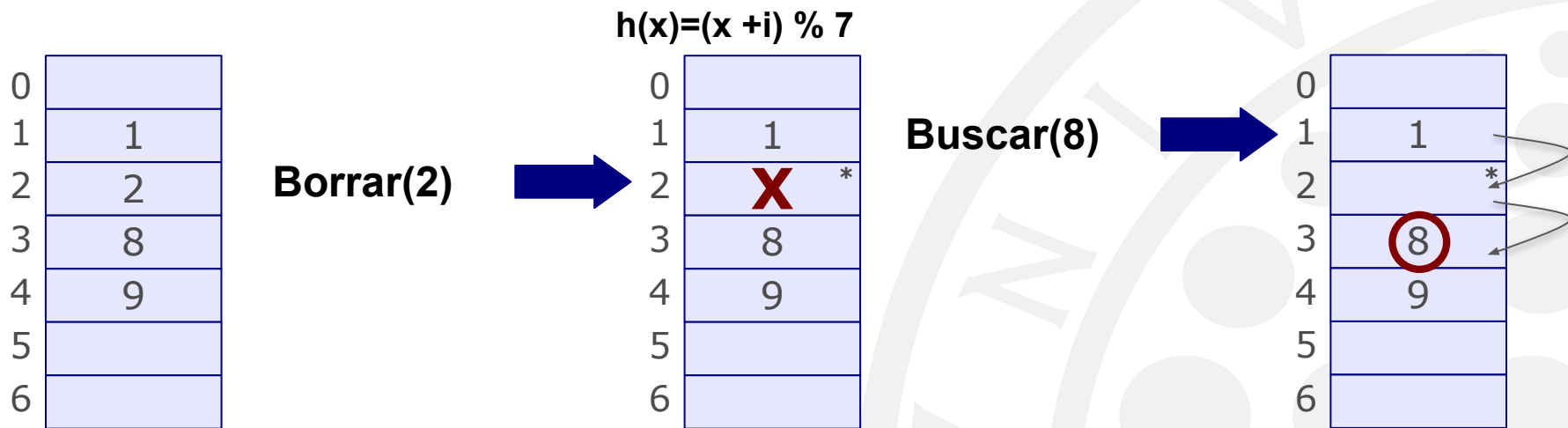
Buscar(8)



0	
1	1
2	2
3	8
4	9
5	
6	

Posiciones vacías y disponibles

- Para resolver el problema, las posiciones borradas se marcarán como **disponibles o borrado**
- Existirán casillas vacías (nunca ocupadas), disponibles (borradas) y ocupadas
- Las búsquedas (y el borrado) sólo pararán al encontrar una casilla vacía



Parte II: Tablas con cubetas

- Definición
- Colisiones en cubetas
- Búsquedas con cubetas

Tablas con cubetas

- Una tabla con **cubetas** o **compartimentos** es una tabla donde cada posición puede contener más de un dato.
- Disminuye el riesgo de reasignación de datos a otras posiciones.

Insertar(1, 2, 8, 11)

$$h(x) = x \% 5$$



0	
1	1 11
2	2
3	8
4	

Colisiones en cubetas

- Con una buena función de dispersión y un tamaño de tabla y cubeta adecuados, los datos deberían ser asignados a la posición que les corresponde.
- Pero al final siempre es necesario disponer de una técnica de resolución de colisiones, como la exploración lineal.

$$h(x) = (x + i) \% 5$$



0	
1	1 11
2	2 6
3	8
4	

Tam. tabla = 5
Tam. cubeta = 2

Insertar (6)

Búsqueda con cubetas

- Usaremos una marca para las **cubetas desbordadas** durante la inserción.
- Continuaremos la búsqueda hasta encontrar el dato o una cubeta que nunca se ha desbordado.

$$h(x) = (x + i) \% 5$$

Buscar (21)



0	
1	1 11
2	2 6
3	8
4	



Rendimiento de una tabla de dispersión

- Una tabla de dispersión cerrada se deteriora notablemente con las inserciones y borrados.
- Aparecen casillas disponibles (o cubetas desbordadas) que alargan las secuencias de búsqueda.
- La solución es la redispersión, aunque con un coste elevado $O(n)$ esperado.
- La dispersión cerrada funciona bien si fundamentalmente se utiliza para búsquedas.
- Si no está bien, no se garantiza el $O(1)$ porque se puede tardar demasiado en insertar un dato.
- Es mucho más sensible que la dispersión abierta a factores de carga altos.

Parte III: Conjuntos y mapas dispersos

- `unordered_map/set`
- `unordered_multimap/multiset`

Conjuntos y mapas dispersos

- Desde C++11 ya existen en STL estándar las clases **unordered_map/set** y **unordered_multimap/multiset** que se implementan como tablas de dispersión cerrada basadas en cubetas.
- Se utilizan de forma similar a los mapas vistos en la Lección 13, e.g. insert, find, operator[], size, etc.
- Disponen de métodos específicos para conocer y controlar el estado de la tabla, e.g. **load_factor**, **max_load_factor**, **bucket_count**, **bucket_size**, **rehash**.

Conjuntos y mapas dispersos

```
#include <unordered_map>
using namespace std;

unordered_map<string,unsigned int> kilometrajes = { {"AAA0000", 115000},
{"BBB1111", 6500} };

kilometrajes.insert( make_pair("CCC2222", 54555) );

cout << "Km. de AAA0000": << kilometrajes["AAA0000"];
unordered_map<string, unsigned int>::iterator it = kilometrajes.find("BBB1111");
if (it != kilometrajes.end() ) {
    cout << "Km. para : " << it->first << " : " << it->second ;
}
cout << "Factor de carga: " << kilometrajes.load_factor();
cout << "Total vehículos: " << kilometrajes.size();
```

Clases de dispersión

- Adicionalmente puede indicarse como parámetro una **clase de dispersión** para el tipo indicado y una clase de comparación de igualdad.

```
class Punto {
    int x, int y;
public:
    Punto (int x1, int y1): x(x1), y(y1) {}
    int leeX() const { return x; }
    int leeY() const { return y; }
};

class DispersionPunto {
public:
    size_t operator() (const Punto &p ) const {
        return p.leeX() * p.leeY();
    }
};

class IgualPunto {
public:
    bool operator() (const Punto &a, const Punto &b) const {
        return a.leeX()==b.leeX() && a.leeY()==b.leeY();
    }
};

unordered_set<Punto, DispersionPunto, IgualPunto> puntos;
```

Consideraciones adicionales en dispersión cerrada (stl)

- Son más eficientes que los mapas si están bien definidas (función $h(x)$ y tamaño).
- No son tan buenas para iterar porque no devuelven los datos ordenados.
- La implementación es dinámica, ampliando el tamaño de la tabla y redispersando cuando el factor de carga se acerca a 1.
- Es conveniente usar la operación `resize()` para indicar desde el principio el tamaño aproximado de la tabla.

Conclusiones

- La dispersión cerrada es más sencilla y utiliza mejor el espacio de la tabla que la dispersión abierta, pero también es susceptible de sufrir mayores problemas de caída de rendimiento.
- El uso de cubetas disminuye el riesgo de colisiones y está especialmente indicado para almacenamiento en memoria secundaria (ficheros).
- Existe una extensión a STL que implementa los conjuntos y mapas mediante tablas de dispersión.

De ahora en adelante...

- La dispersión cerrada hace uso de memoria secundario en ficheros.
- Disminuye el número de colisiones aunque no los elimina del todo.

Lección 16: Grafos